Authentifier les clients keycloak via un certificat x509

https://github.com/0b11stan/poc-keycloak-x509

Keycloak est un serveur d'authentification très réputé auprès des entreprises de grandes taille :

- très grand éventail de fonctionnalité
- excellentes performances
- sous license libre
- sponsorisé par RedHat
- permet la mise en place d'un SSO
- fonctionne pour les interractions humain/machine ou machine/machine

Fonctionnement général du SSO:

1. le client s'identifie et s'authentie auprès d'un serveur A (keycloak)
2. le serveur délivre un jeton unique au client (jwt)
3. le client peut joindre ce jeton à toutes ses requêtes pour s'authentifier
4. les autres serveurs n'ont qu'a vérifier auprès du serveur A la validité du jeton

Par défaut, la première étape (l'authentification du client) se fait à l'aide d'un couple `CLIENT_ID`/`CLIENT_SECRET`. Cette méthode d'authentification n'est pas parfaite puisqu'une donnée "secrete" doit transiter sur le réseau (`CLIENT_SECRET`). Cet article montre comment utiliser la fonctionnalité d'authentification TLS mutuelle sur keycloak pour renforcer la sécurité et limiter la circulation de secrets sensibles.

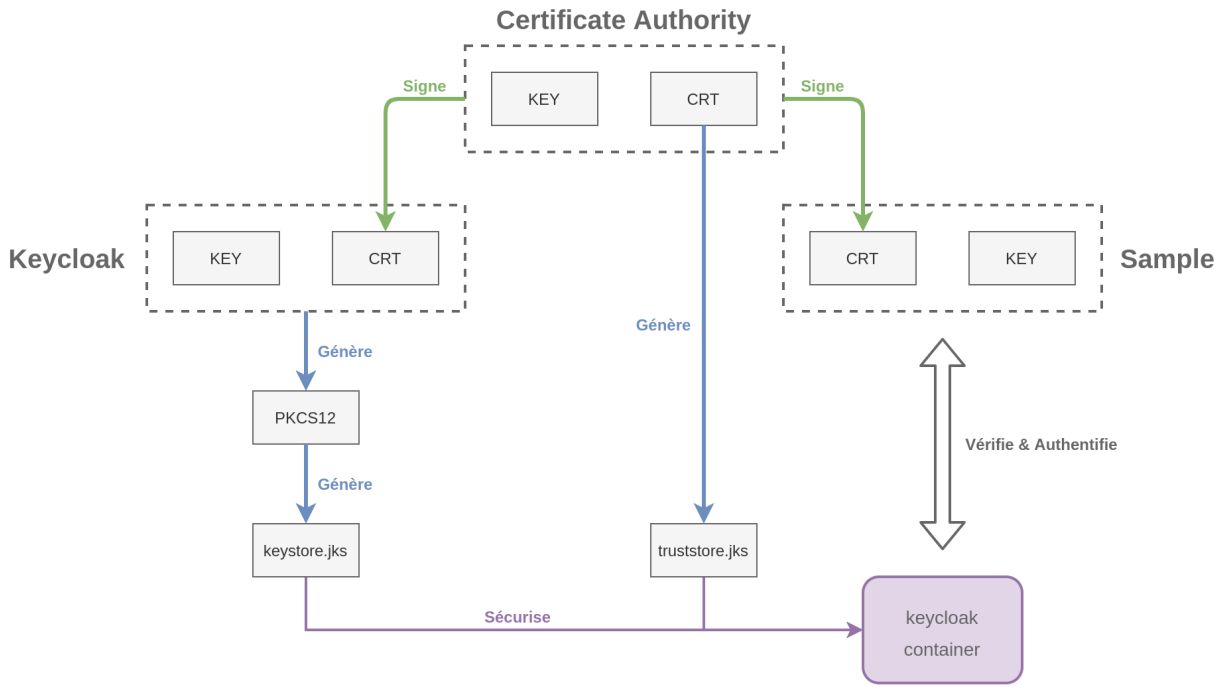
Cet article s'intéresse surtout à l'authentification d'un composant "machine" mais la méthode peut être facilement étendue pour l'authentification d'un composant "utilisateur".

Mise en place du lab

Le poc est un dépôt GIT qui contient l'arborescence suivante:

```
.
|-- ca
|   `-- build_certs.sh
|-- configuration
|   |-- keycloak-add-user.json
|   `-- standalone-ha.xml
|-- docker-compose.yml
|-- init_realm.sh
|-- README.md
`-- realm_internal.json
```

Avant toute chose, il faut générer une PKI suivant ce schéma.



Une autorité de certification va signer un certificat pour keycloak et un autre pour notre client de test (qu'on appellera dans tous nos POC: *sample*). Pour que le serveur Keycloak puisse correctement manipuler nos documents cryptographiques, on doit les intégrer à des **Java Key Store**. C'est un format propriétaire équivalent à PKCS12 qui est supporté par wildfly. On va donc fournir au démarrage de keycloak deux **JKS**:

- `truststore.jks` qui contient le certificat de notre autorité de certification
- `keystore.jks` qui contient le couple de clefs public/privé de keycloak

Pour générer cette PKI il suffit de jouer le script dans le répertoire `ca`. Ce script ajoute les deux keystore java vu précédemment au répertoire `configuration`.

```
pushd ca && ./build_certs.sh && popd
```

Le docker-compose permet de démarrer un keycloak en local avec la `./configuration` appropriée.

```
docker-compose up -d
```

Une fois que Keycloak a démarré, un script permet de créer un royaume `internal` pré-configuré à partir d'un fichier de presseed. (Pour vérifier si keycloak à démarré: `docker-compose logs | grep 'Admin console listening'`)

Ce royaume contient simplement un client `SAMPLE-API`.

```
./init_realm.sh
```

On se trouve donc dans le cas nominal ou un client peut se connecter à l'aide d'un couple `CLIENT_ID/CLIENT_SECRET`.

```
client_id=SAMPLE-API
client_secret=e1efb404-a846-42e9-a8dd-836e841dc908
url="https://localhost:8443/auth/realms/internal/protocol/openid-connect/token"
curl --silent --insecure --location --request POST $url \
  --header 'Content-Type: application/x-www-form-urlencoded' \
  --data-urlencode "client_id=$client_id" \
  --data-urlencode "client_secret=$client_secret" \
  --data-urlencode 'grant_type=client_credentials'
```

Output :

```
{
  "access_token": "eyJhbG [...] bLCA",
  "expires_in": 300,
  "refresh_expires_in": 0,
  "token_type": "Bearer",
  "not-before-policy": 0,
  "scope": "email profile"
}
```

Execution du POC

Pour que nos JKS soient pris en compte, et que l'authentification TLS mutuelle soit activée on a dû modifier la configuration de base. Les commandes suivantes permettent de constater ces modifications pour la version 14 de keycloak :

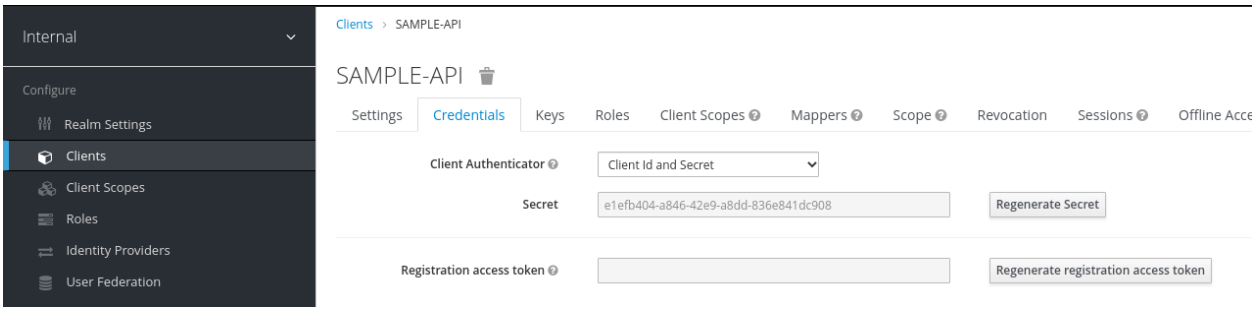
```
docker run --rm bitnami/keycloak:14.0.0 \
  cat /opt/bitnami/keycloak/standalone/configuration/standalone-ha-default.xml 2> /dev/null \
  | sed 's/$/\r/' > /tmp/standalone-ha.xml
diff --color /tmp/standalone-ha.xml configuration/standalone-ha.xml
```

Output :

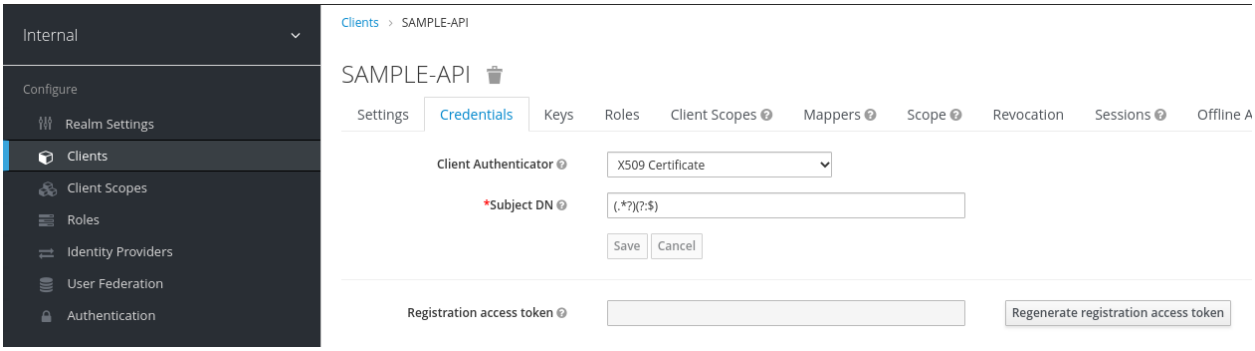
```
1d0
<
48c47,51
<                         <keystore path="application.keystore" relative-to="jboss.server.config.dir" ke
---
>                         <keystore
>                             path="keystore.jks"
>                             relative-to="jboss.server.config.dir"
>                             keystore-password="azerty"
>                             key-password="azerty"/>
58a62,75
>             <security-realm name="ssl-realm">
>                 <server-identities>
>                     <ssl>
>                         <keystore path="keystore.jks"
>                                   relative-to="jboss.server.config.dir"
>                                   keystore-password="azerty"/>
>                     </ssl>
>                 </server-identities>
>                 <authentication>
>                     <truststore path="truststore.jks"
>                                 relative-to="jboss.server.config.dir"
>                                 keystore-password="azerty"/>
>                 </authentication>
>             </security-realm>
632c649,654
<                 <https-listener name="https" socket-binding="https" security-realm="ApplicationRealm"
---
>                 <https-listener
>                     name="https"
>                     socket-binding="https"
>                     enable-http2="true"
```

```
>                     security-realm="ssl-realm"
>                     verify-client="REQUESTED"/>
677c699
< </server>
\ No newline at end of file
---
> </server>
```

Se rendre sur le keycloak local (authentification : `admin:admin`) puis dans les credentials du client `SAMPLE-API`.



Choisir `x509 Certificate` comme `Client Authenticator` et spécifier l'expression régulière suivante dans le champ `Subject DN`: `(.*?)(?:$)`. Cette expression régulière match la totalité du champ correspondant dans le certificat du **client TLS** pour reconnaître un **client Keycloak**.



Tester l'autentification avec certificat

Pour obtenir un token de keycloak, plus besoin de fournir un secret en clair. Il suffit d'utiliser un certificat et une clef signée par une CA en qui keycloak fait confiance. L'authentification se déroule lors du handshake TLS.

```
curl https://localhost:8443/auth/realms/internal/protocol/openid-connect/token \
  --request POST \
  --header 'Content-Type: application/x-www-form-urlencoded' \
  --data-urlencode 'client_id=SAMPLE-API' \
  --data-urlencode 'grant_type=client_credentials' \
  --cacert ca/ca.crt.pem \
  --cert ca/sample.crt.pem \
  --key ca/sample.key.pem
```

Output :

```
{
  "access_token": "eyJhbG [...] elKOhIww",
  "expires_in": 300,
  "refresh_expires_in": 0,
```

```
  "token_type": "Bearer",
  "not-before-policy": 0,
  "scope": "email profile"
}
```

Le secret n'est plus statique. Si la clef privée du client est compromise, il suffira pour l'équipe en charge du composant de régénérer une nouvelle clef privé accompagnée d'une CSR. Naturellement, la CA générée ici est pauvrement sécurisée par un mot de passe faible. Dans un environnement de production, on souhaitera s'attacher à une PKI plus sérieuse et qui profite d'un processus de vérification rodé.

L'utilisation du chiffrement asymétrique permet de ne jamais dévoiler de secrets sur le réseau, ce qui restreint encore la surface d'attaque.

Enfin, le certificat à l'avantage de fortement authentifier son porteur et sur une période de temps limité. Comme on peut le voir dans le script `build_certs.sh`, le `CLIENT_ID` est utiliser dans le champ **CN** du certificat pour reconnaître le composant originaire de la demande, mais on pourrait imaginer que keycloak cherche des informations complémentaires comme une adresse ip ou un nom d'hôte pour correctement identifier le composant.